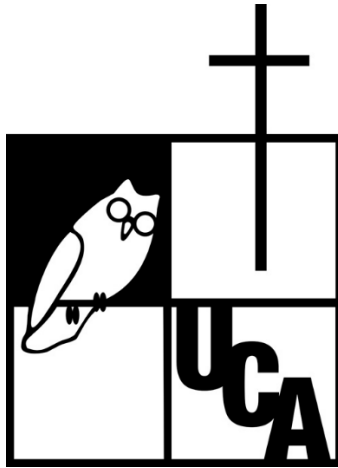


UNIVERSIDAD CENTROAMERICANA
JOSÉ SIMEÓN CAÑAS



PROYECTO FINAL

Bases de Datos

Sistema de Reservas de Hotel

Escenario B – Hotel y reservaciones

CATEDRATICO

Ing. James Edward Humberstone Morales

Estudiantes

Nombre:

Adán Mauricio Larios Alfaro	00099123
Angel Gustavo Monterrosa Cisneros	00048825
Samuel André Morán Escobar	00133225
Luis José Murcia Portillo	00249025
Willians Anderson Nájera Sánchez	00041625

Contenido

1. Descripción General del Proyecto	3
1.1 Objetivos	3
Descripción del Escenario y Reglas de Negocio	4
2.1 Contexto	4
2.2 Reglas de negocio Un huésped puede realizar múltiples reservaciones a lo largo del tiempo.	4
2.3 Supuestos y decisiones de diseño	5
3. Modelo Entidad–Relación	6
3.1 Diagrama Entidad–Relación	6
3.2 Entidades y atributos	6
3.3 Relaciones y cardinalidades	7
4. Esquema Relacional Normalizado	8
4.1 Esquema relacional	8
4.2 Justificación de la normalización	8
5. Diccionario de Datos	9
6. Consultas SQL Frecuentes	12
6.1 Habitaciones disponibles en un rango de fechas.....	12
6.2 Huéspedes con mayor gasto histórico	13
6.3 Servicios más consumidos por tipo de habitación	13
6.4 Tasa de ocupación mensual por tipo de habitación	14
6.5 Ingresos totales por mes (año en curso).....	16
7. Programación de la Base de Datos.....	17
7.1 Función Habitaciones disponibles (no doble reserva)	17
7.2 Tasa Ocupación	18
7.3 Función auxiliar: total de una estancia.....	19
7.4 Procedimiento almacenado: check-out.....	20
8. Instrucciones de Ejecución.....	21
8.1 Archivos entregables	21

1. Descripción General del Proyecto

El presente documento describe el diseño e implementación de una base de datos relacional para la gestión de un hotel, desarrollada como proyecto final del curso de Bases de Datos. La solución abarca todo el ciclo de vida del diseño: el modelo conceptual Entidad–Relación en notación Chen, su transformación a un esquema relacional normalizado hasta la Tercera Forma Normal (3FN), la implementación en PostgreSQL 17 mediante sentencias DDL con restricciones de integridad, la carga y manipulación de datos con DML, un conjunto de consultas SQL que responden a las necesidades del negocio y la lógica procedimental (funciones, procedimientos almacenados y disparadores). El sistema corresponde al Escenario B propuesto: la administración de habitaciones, huéspedes, reservaciones, servicios adicionales y la facturación de las estancias.

1.1 Objetivos

- Diseñar el modelo conceptual de datos en notación Chen.
- Transformar el modelo conceptual en un esquema relacional normalizado (3FN).
- Implementar la base de datos en PostgreSQL con restricciones de integridad.
- Poblar y manipular los datos mediante sentencias DML.
- Formular consultas SQL básicas y avanzadas para el negocio.
- Desarrollar lógica procedimental: transacciones, funciones, procedimientos y triggers.
- Documentar técnicamente el diseño y la implementación

Descripción del Escenario y Reglas de Negocio

2.1 Contexto

Un hotel necesita administrar sus habitaciones, huéspedes, reservaciones, servicios adicionales y la facturación de las estancias. Un huésped puede realizar múltiples reservaciones; cada reservación cubre una habitación durante un rango de fechas. Durante la estancia el huésped puede consumir servicios adicionales (restaurante, lavandería, spa, etc.) y, al hacer check-out, se genera una factura consolidada.

2.2 Reglas de negocio

Un huésped puede realizar múltiples reservaciones a lo largo del tiempo.

1. Una reservación cubre una habitación específica durante un rango de fechas [fecha_inicio, fecha_fin).
2. Una habitación no puede estar reservada dos veces en el mismo período (regla garantizada mediante un disparador).
3. Durante la estancia, el huésped puede consumir varios servicios adicionales.
4. Cada reservación que se concreta genera una estancia (check-in / check-out) que es registrada por un empleado.
5. Al realizar el check-out se calcula el total (hospedaje + servicios + impuestos) y se genera una única factura por estancia.
6. Un empleado pertenece a un hotel; un hotel posee varias habitaciones (clasificadas por tipo) y ofrece varios servicios.

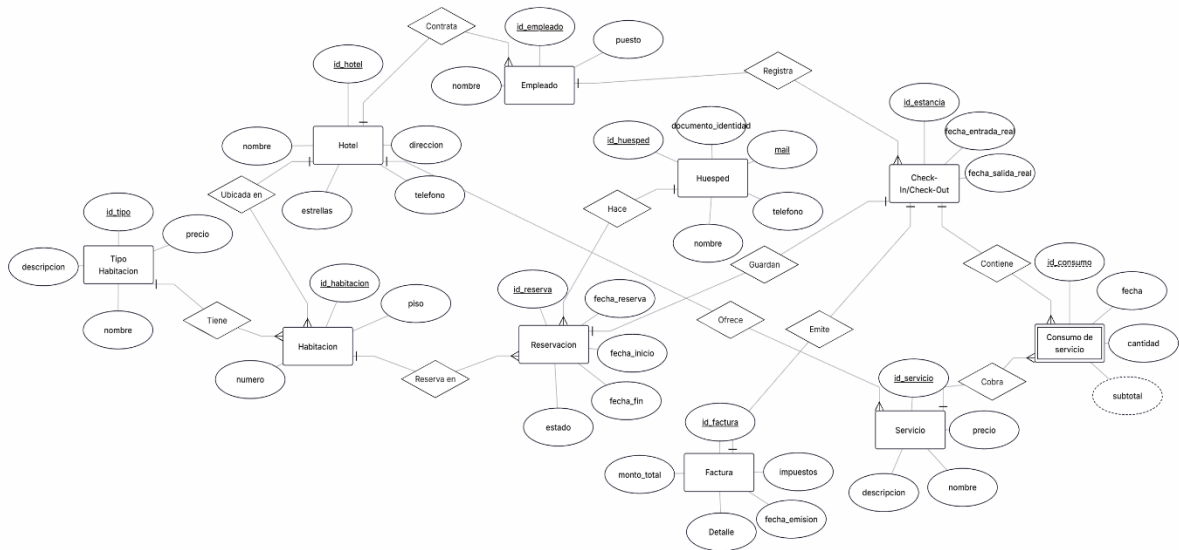
2.3 Supuestos y decisiones de diseño

- La reservación referencia una habitación específica (no solo el tipo).
- Esto permite validar con exactitud la regla de "no doble reserva".
- Se utilizan intervalos semiabiertos [inicio, fin): el día de salida queda disponible para que otro huésped ingrese ese mismo día.
- La habitación asociada a una estancia se deriva de su reservación; no se almacena de nuevo en la estancia, evitando una dependencia transitiva (3FN).
- En la factura, monto_total incluye los impuestos y el campo impuestos guarda el monto del impuesto. La tasa por defecto es 12 % y es parametrizable en el procedimiento de check-out.
- Las claves primarias son surrogadas (columnas IDENTITY). El número de habitación es único dentro de cada hotel (restricción UNIQUE(id_hotel, numero)). subtotal (de consumo) y monto_total (de factura) son valores derivados; subtotal se mantiene automáticamente con un trigger y monto_total se calcula al hacer check-out.
- "Aunque a nivel conceptual se identificó la entidad Hotel (como se ve en los diagramas), al tratarse del Escenario B correspondiente a un único establecimiento, en el esquema físico (script DDL) se tomó la decisión de diseño de omitir la tabla Hotel y sus claves foráneas para evitar redundancia de datos."

3. Modelo Entidad–Relación

3.1 Diagrama Entidad–Relación

Figura 1. Diagrama Entidad–Relación en notación Chen. El archivo editable se incluye como ProyectoBD_corregido.erdplus (abrible en ERDPlus).



3.2 Entidades y atributos

Columna1	Atributos (clave primaria subrayada)
Hotel	id_hotel (PK), nombre, direccion, telefono, estrellas
Tipo Habitacion	id_tipo (PK), nombre, descripcion, precio
Habitacion	id_habitacion (PK), numero, piso
Huesped	id_huesped (PK), documento_identidad, nombre, mail, telefono
Empleado	id_empleado (PK), nombre, puesto
Servicio	id_servicio (PK), nombre, descripcion, precio
Reservacion	id_reserva (PK), fecha_reserva, fecha_inicio, fecha_fin, estado
Check-In/Check-Out (Estancia)	id_estancia (PK), fecha_entrada_real, fecha_salida_real
Consumo de servicio (debil)	id_consumo (PK parcial), fecha, cantidad, subtotal (derivado)
Factura	id_factura (PK), fecha_emision, monto_total, impuestos, detalle

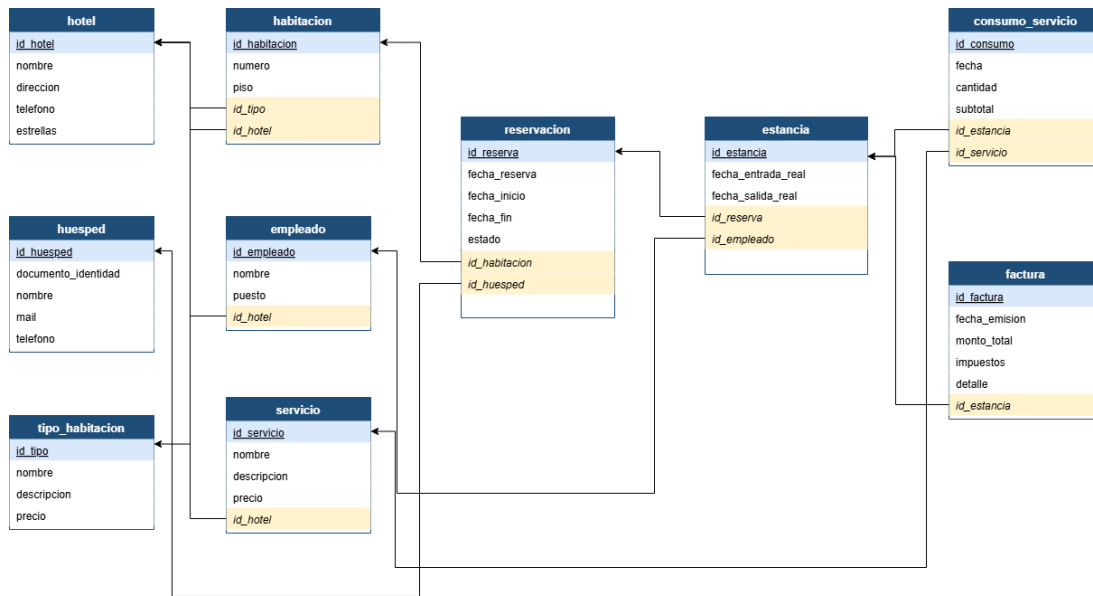
3.3 Relaciones y cardinalidades

Relacion	Entidad A	Card.	Entidad B	Card.2
Contrata	Empleado	N	Hotel	1
Ubicada en	Habitacion	N	Hotel	1
Tiene	Tipo Habitacion	1	Habitacion	N
Ofrece	Servicio	N	Hotel	1
Hace	Huesped	1	Reservacion	N
Reserva en	Habitacion	1	Reservacion	N
Guardan	Reservacion	1	Check-In/Check-Out	1
Registra	Empleado	1	Check-In/Check-Out	N
Contiene	Check-In/Check-Out	1	Consumo de servicio	N
Cobra	Consumo de servicio	N	Servicio	1
Emite	Factura	1	Check-In/Check-Out	1

4. Esquema Relacional Normalizado

4.1 Esquema relacional

Figura 2. Diagrama relacional (modelo físico): tablas, columnas y relaciones entre claves foráneas y primarias.



4.2 Justificación de la normalización

Primera Forma Normal (1FN):

Todos los atributos son atómicos: cada celda almacena un único valor y no existen grupos repetitivos ni atributos multivaluados. Por ejemplo, los servicios consumidos no se guardan como una lista dentro de la estancia, sino como filas independientes en consumo_servicio.

Segunda Forma Normal (2FN):

El esquema está en 1FN y todas las tablas usan una clave primaria simple (un solo atributo), por lo que no pueden existir dependencias parciales: cada atributo no clave depende de la totalidad de la clave primaria.

Tercera Forma Normal (3FN):

No existen dependencias transitivas: ningún atributo no clave depende de otro atributo no clave. Los datos del tipo de habitación residen únicamente en tipo_habitacion y se referencian con id_tipo; la habitación de una estancia no se almacena, sino que se obtiene a través de la reservación; y la factura conserva sus montos como un registro histórico del comprobante (valores ya calculados al momento del check-out).

5. Diccionario de Datos

Tipo habitación

Campo	Tipo	Nulo	Clave	Descripcion
id_tipo	INT (IDENTITY)	No	PK	Identificador del tipo
nombre	VARCHAR(50)	No	UQ	Nombre del tipo (Individual, Doble, Suite)
descripcion	VARCHAR(200)	Si		Descripcion del tipo
precio	NUMERIC(10,2)	No		Tarifa por noche (CHECK >= 0)

Habitación

Campo	Tipo	Nulo	Clave	Descripcion
id_habitacion	INT (IDENTITY)	No	PK	Identificador de la habitacion
numero	VARCHAR(10)	No	UQ*	Numero de habitacion (unico por hotel)
piso	SMALLINT	Si		Piso donde se ubica
id_tipo	INT	No	FK	Tipo de habitacion

Huésped

Campo	Tipo	Nulo	Clave	Descripcion
id_huesped	INT (IDENTITY)	No	PK	Identificador del huesped
documento_identidad	VARCHAR(20)	No	UQ	Documento de identidad
nombre	VARCHAR(100)	No		Nombre completo
mail	VARCHAR(120)	Si	UQ	Correo electronico
telefono	VARCHAR(20)	Si		Telefono de contacto

Empleado

Campo	Tipo	Nulo	Clave	Descripcion
id_empleado	INT (IDENTITY)	No	PK	Identificador del empleado
nombre	VARCHAR(100)	No		Nombre completo
puesto	VARCHAR(50)	No		Puesto desempenado

Servicio

Campo	Tipo	Nulo	Clave	Descripcion
id_servicio	INT (IDENTITY)	No	PK	Identificador del servicio
nombre	VARCHAR(50)	No		Nombre del servicio
descripcion	VARCHAR(200)	Si		Descripcion del servicio
precio	NUMERIC(10,2)	No		Precio unitario (CHECK >= 0)

Reservación

Campo	Tipo	Nulo	Clave	Descripcion
id_reserva	INT (IDENTITY)	No	PK	Identificador de la reservacion
fecha_reserva	DATE	No		Fecha en que se hizo la reserva
fecha_inicio	DATE	No		Inicio del hospedaje
fecha_fin	DATE	No		Fin del hospedaje (CHECK > inicio)
estado	VARCHAR(20)	No		CONFIRMADA / CANCELADA / FINALIZADA
id_huesped	INT	No	FK	Huesped que reserva
id_habitacion	INT	No	FK	Habitacion reservada

Estancia(check_in_check_out)

Campo	Tipo	Nulo	Clave	Descripcion
id_estancia	INT (IDENTITY)	No	PK	Identificador de la estancia
fecha_entrada_real	DATE	No		Fecha real de check-in
fecha_salida_real	DATE	Si		Fecha real de check-out (NULL si abierta)
id_reserva	INT	No	FK/UQ	Reservacion asociada (1:1)
id_empleado	INT	No	FK	Empleado que registra el check-in/out

Consumo servicio

Campo	Tipo	Nulo	Clave	Descripcion
id_consumo	INT (IDENTITY)	No	PK	Identificador del consumo
fecha	DATE	No		Fecha del consumo
cantidad	INT	No		Unidades consumidas (CHECK > 0)
subtotal	NUMERIC(10,2)	Si		Derivado: cantidad x precio (trigger)
id_estancia	INT	No	FK	Estancia donde se consumo
id_servicio	INT	No	FK	Servicio consumido

Factura

Campo	Tipo	Nulo	Clave	Descripcion
id_factura	INT (IDENTITY)	No	PK	Identificador de la factura
fecha_emision	DATE	No		Fecha de emision
monto_total	NUMERIC(12,2)	No		Total con impuestos (CHECK >= 0)
impuestos	NUMERIC(12,2)	No		Monto del impuesto
detalle	TEXT	Si		Desglose del comprobante
id_estancia	INT	No	FK/UQ	Estancia facturada (1:1)

6. Consultas SQL Frecuentes

A continuación se documentan las consultas más relevantes para el negocio.

6.1 Habitaciones disponibles en un rango de fechas

Devuelve las habitaciones que no tienen ninguna reserva activa que se traslape con el rango solicitado (en el ejemplo, del 05 al 10 de junio de 2026).

```
-----
SELECT  h.id_habitacion,
        h.numero,
        th.nombre      AS tipo,
        th.precio      AS precio_noche
FROM    habitacion h
JOIN    tipo_habitacion th ON th.id_tipo = h.id_tipo
WHERE   NOT EXISTS (
        SELECT 1
        FROM    reservacion r
        WHERE   r.id_habitacion = h.id_habitacion
              AND r.estado IN ('PENDIENTE', 'CONFIRMADA', 'CHECK_IN')
              AND daterange(r.fecha_inicio, r.fecha_fin, '[')
                  && daterange(
DATE '2026-06-05', DATE '2026-06-10', '[')
        )
ORDER BY h.piso, h.numero;
```

Resultado de muestra:

	123 id_habitacion	AZ numero	AZ tipo	123 precio_noche
3	3	103	Doble Estandar	28.000
4	4	104	Doble Superior	35.000
5	5	105	Simple	18.000
6	6	201	Doble Estandar	28.000
7	7	202	Suite Junior	65.000
8	8	203	Doble Superior	35.000
9	9	204	Suite Premium	95.000
10	10	205	Simple	18.000
11	11	301	Doble Estandar	28.000
12	12	302	Doble Superior	35.000
13	13	303	Doble Superior	35.000
14	14	304	Triple	42.000
15	15	305	Suite Junior	65.000
16	16	401	Suite Junior	65.000
17	17	402	Suite Premium	95.000
18	18	403	Suite Premium	95.000
19	19	404	Simple	18.000
20	20	405	Doble Estandar	28.000
21	21	501	Simple	18.000
22	22	502	Simple	18.000
23	23	503	Doble Estandar	28.000
24	24	504	Doble Estandar	28.000
25	25	505	Doble Superior	35.000
26	26	601	Triple	42.000
27	27	602	Suite Junior	65.000
28	28	603	Suite Premium	95.000
29	29	604	Doble Estandar	28.000
30	30	605	Doble Superior	35.000

6.2 Huéspedes con mayor gasto histórico

Suma el monto de todas las facturas asociadas a cada huésped y los ordena de mayor a menor.

```
SELECT  hu.id_huesped,
        hu.nombre,
        hu.documento_identidad,
        COUNT(f.id_factura)           AS estancias,
        SUM(f.monto_total)           AS gasto_total
FROM    huesped hu
JOIN    reservacion r                ON r.id_huesped = hu.id_huesped
JOIN    check_in_check_out e        ON e.id_reserva = r.id_reserva
JOIN    factura f                   ON f.id_estancia = e.id_estancia
GROUP BY hu.id_huesped, hu.nombre, hu.documento_identidad
ORDER BY gasto_total DESC
LIMIT 10;
```

Resultado de muestra:

	123 id_huesped	AZ nombre	AZ documento_identidad	123 estancias	123 gasto_total
1	6	Roberto Martinez	CD789012	1	872.410
2	5	Lucia Fernandez	40234567	1	484.000
3	3	Carlos Lopez	35987654	1	469.480
4	1	Juan Perez	30123456	1	232.925
5	8	Diego Morales	CL987654	1	183.920
6	2	Maria Garcia	28765432	1	156.090
7	7	Sofia Rodriguez	32456789	1	124.630
8	4	Ana Silva	AB123456	1	114.345

6.3 Servicios más consumidos por tipo de habitación

Relaciona cada consumo con el tipo de la habitación donde se hospedó el huésped que lo generó, agrupando por tipo y servicio.

```
SELECT  th.nombre           AS tipo_habitacion,
        s.nombre           AS servicio,
        SUM(cs.cantidad)    AS unidades_consumidas,
        SUM(cs.subtotal)    AS ingresos_servicio
FROM    consumo_servicio cs
JOIN    servicio s          ON s.id_servicio = cs.id_servicio
JOIN    check_in_check_out e ON e.id_estancia = cs.id_estancia
JOIN    habitacion h        ON h.id_habitacion = e.id_habitacion
JOIN    tipo_habitacion th  ON th.id_tipo = h.id_tipo
GROUP BY th.nombre, s.nombre
ORDER BY th.nombre, unidades_consumidas DESC;
```

Resultado de muestra:

	AZ tipo_habitacion	AZ servicio	123 unidades_consumidas	123 ingresos_servicio
1	Doble Estandar	Desayuno buffet	5	22.500
2	Doble Estandar	Consumo minibar	4	10.000
3	Doble Estandar	Tragos premium	3	19.500
4	Doble Estandar	Cena gourmet	2	24.000
5	Doble Estandar	Lavado de ropa	2	6.000
6	Doble Superior	Desayuno buffet	2	9.000
7	Doble Superior	Masaje relax 60	1	15.000
8	Suite Junior	Desayuno buffet	4	18.000
9	Suite Junior	Cena gourmet	4	48.000
10	Suite Junior	Masaje relax 60	3	45.000
11	Suite Junior	Spa - circuito	2	18.000
12	Suite Junior	Masaje deportivo	2	33.000
13	Suite Premium	Desayuno buffet	9	40.500
14	Suite Premium	Estacionamiento	7	24.500
15	Suite Premium	Cena gourmet	2	24.000
16	Triple	Consumo minibar	5	12.500
17	Triple	Desayuno buffet	3	13.500

6.4 Tasa de ocupación mensual por tipo de habitación

Expande cada reserva en sus noches individuales y calcula, por mes y tipo, las noches ocupadas frente a las noches disponibles (habitaciones del tipo × días del mes).

```

WITH meses AS (
    SELECT generate_series(
        date_trunc('year', CURRENT_DATE)::date,
        date_trunc('year', CURRENT_DATE)::date + INTERVAL '11 months',
        INTERVAL '1 month'
    )::date AS inicio_mes
),
mes_rango AS (
    SELECT inicio_mes,
        (inicio_mes + INTERVAL '1 month')::date AS fin_mes,
        EXTRACT(DAY FROM (inicio_mes + INTERVAL '1 month - 1 day'))::int AS dias_mes
    FROM meses
),
ocupacion AS (
    SELECT mr.inicio_mes,
        th.id_tipo,
        th.nombre,
        COALESCE(SUM(
            GREATEST(
                0,
                LEAST(r.fecha_fin, mr.fin_mes)
                - GREATEST(r.fecha_inicio, mr.inicio_mes)
            ), 0) AS noches_reservadas
    FROM mes_rango mr
    CROSS JOIN tipo_habitacion th
    LEFT JOIN habitacion h ON h.id_tipo = th.id_tipo
    LEFT JOIN reservacion r ON r.id_habitacion = h.id_habitacion
        AND r.estado IN ('CONFIRMADA', 'CHECK_IN', 'CHECK_OUT')
        AND r.fecha_inicio < mr.fin_mes
        AND r.fecha_fin > mr.inicio_mes
    GROUP BY mr.inicio_mes, th.id_tipo, th.nombre
),
capacidad AS (
    SELECT th.id_tipo,
        COUNT(h.id_habitacion) AS cant_habitaciones
    FROM tipo_habitacion th
    LEFT JOIN habitacion h ON h.id_tipo = th.id_tipo
    GROUP BY th.id_tipo
)
SELECT to_char(o.inicio_mes, 'YYYY-MM') AS mes,
    o.nombre AS tipo,
    c.cant_habitaciones,
    o.noches_reservadas,
    c.cant_habitaciones * mr.dias_mes AS noches_disponibles,
    ROUND(
        100.0 * o.noches_reservadas
        / NULLIF(c.cant_habitaciones * mr.dias_mes, 0),
        2) AS tasa_ocupacion_pct
FROM ocupacion o
JOIN capacidad c ON c.id_tipo = o.id_tipo
JOIN mes_rango mr ON mr.inicio_mes = o.inicio_mes
ORDER BY mes, tipo;

```

Resultado de muestra:

	AZ mes	AZ tipo	123 cant_habitaciones	123 noches_reservadas	123 noches_disponibles	123 tasa_ocupacion_pct
1	2026-01	Doble Estandar	8	222	248	89,52
2	2026-01	Doble Superior	6	186	186	100
3	2026-01	Simple	6	186	186	100
4	2026-01	Suite Junior	4	124	124	100
5	2026-01	Suite Premium	4	124	124	100
6	2026-01	Triple	2	62	62	100
7	2026-02	Doble Estandar	8	224	224	100
8	2026-02	Doble Superior	6	143	168	85,12
9	2026-02	Simple	6	168	168	100
10	2026-02	Suite Junior	4	89	112	79,46
11	2026-02	Suite Premium	4	112	112	100
12	2026-02	Triple	2	56	56	100
13	2026-03	Doble Estandar	8	220	248	88,71
14	2026-03	Doble Superior	6	186	186	100
15	2026-03	Simple	6	186	186	100
16	2026-03	Suite Junior	4	98	124	79,03
17	2026-03	Suite Premium	4	124	124	100
18	2026-03	Triple	2	62	62	100
19	2026-04	Doble Estandar	8	213	240	88,75
20	2026-04	Doble Superior	6	180	180	100
21	2026-04	Simple	6	180	180	100
22	2026-04	Suite Junior	4	120	120	100
23	2026-04	Suite Premium	4	97	120	80,83
24	2026-04	Triple	2	33	60	55
25	2026-05	Doble Estandar	8	248	248	100
26	2026-05	Doble Superior	6	158	186	84,95
27	2026-05	Simple	6	186	186	100
28	2026-05	Suite Junior	4	97	124	78,23
29	2026-05	Suite Premium	4	97	124	78,23
30	2026-05	Triple	2	62	62	100
31	2026-06	Doble Estandar	8	213	240	88,75
32	2026-06	Doble Superior	6	180	180	100
33	2026-06	Simple	6	180	180	100
34	2026-06	Suite Junior	4	120	120	100
35	2026-06	Suite Premium	4	95	120	79,17
36	2026-06	Triple	2	60	60	100
37	2026-07	Doble Estandar	8	248	248	100
38	2026-07	Doble Superior	6	186	186	100
39	2026-07	Simple	6	186	186	100
40	2026-07	Suite Junior	4	124	124	100
41	2026-07	Suite Premium	4	124	124	100
42	2026-07	Triple	2	62	62	100
43	2026-08	Doble Estandar	8	248	248	100
44	2026-08	Doble Superior	6	186	186	100
45	2026-08	Simple	6	186	186	100
46	2026-08	Suite Junior	4	124	124	100
47	2026-08	Suite Premium	4	124	124	100
48	2026-08	Triple	2	62	62	100
49	2026-09	Doble Estandar	8	240	240	100
50	2026-09	Doble Superior	6	180	180	100
51	2026-09	Simple	6	180	180	100
52	2026-09	Suite Junior	4	120	120	100
53	2026-09	Suite Premium	4	120	120	100
54	2026-09	Triple	2	60	60	100
55	2026-10	Doble Estandar	8	248	248	100
56	2026-10	Doble Superior	6	186	186	100
57	2026-10	Simple	6	186	186	100
58	2026-10	Suite Junior	4	124	124	100
59	2026-10	Suite Premium	4	124	124	100
60	2026-10	Triple	2	62	62	100
61	2026-11	Doble Estandar	8	240	240	100
62	2026-11	Doble Superior	6	180	180	100
63	2026-11	Simple	6	180	180	100
64	2026-11	Suite Junior	4	120	120	100
65	2026-11	Suite Premium	4	120	120	100
66	2026-11	Triple	2	60	60	100
67	2026-12	Doble Estandar	8	248	248	100
68	2026-12	Doble Superior	6	186	186	100
69	2026-12	Simple	6	186	186	100
70	2026-12	Suite Junior	4	124	124	100
71	2026-12	Suite Premium	4	124	124	100
72	2026-12	Triple	2	62	62	100

6.5 Ingresos totales por mes (año en curso)

Agrupar las facturas del año actual por mes de emisión y suma sus montos.

```
SELECT  to_char(date_trunc('month', f.fecha_emision), 'YYYY-MM') AS mes,
        COUNT(*) AS cantidad_facturas,
        SUM(f.monto_total - f.impuestos) AS ingresos_netos,
        SUM(f.impuestos) AS impuestos_cobrados,
        SUM(f.monto_total) AS ingresos_totales
FROM    factura f
WHERE   EXTRACT(YEAR FROM f.fecha_emision) = EXTRACT(YEAR FROM CURRENT_DATE)
GROUP BY date_trunc('month', f.fecha_emision)
ORDER BY mes;
```

Resultado de muestra:

	A-Z mes	123 cantidad_facturas	123 ingresos_netos	123 impuestos_cobrados	123 ingresos_totales
1	2026-01	1	192.500	40.425	232.925
2	2026-02	2	517.000	108.570	625.570
3	2026-03	2	494.500	103.845	598.345
4	2026-04	3	976.000	204.960	1.180.960

7. Programación de la Base de Datos

7.1 Función Habitaciones disponibles (no doble reserva)

Devuelve las habitaciones libres en un rango de fechas.

```
CREATE OR REPLACE FUNCTION fn_habitaciones_disponibles (  
    p_fecha_inicio DATE,  
    p_fecha_fin    DATE  
)  
RETURNS TABLE (  
    id_habitacion INT,  
    numero        VARCHAR,  
    tipo          VARCHAR,  
    precio_noche  NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    IF p_fecha_fin <= p_fecha_inicio THEN  
        RAISE EXCEPTION 'fecha_fin (%) debe ser posterior a fecha_inicio (%)',  
            p_fecha_fin, p_fecha_inicio;  
    END IF;  
  
    RETURN QUERY  
    SELECT h.id_habitacion,  
        h.numero,  
        th.nombre,  
        th.precio  
    FROM habitacion h  
    JOIN tipo_habitacion th ON th.id_tipo = h.id_tipo  
    WHERE NOT EXISTS (  
        SELECT 1  
        FROM reservacion r  
        WHERE r.id_habitacion = h.id_habitacion  
            AND r.estado IN ('PENDIENTE', 'CONFIRMADA', 'CHECK_IN')  
            AND daterange(r.fecha_inicio, r.fecha_fin, '[')  
                && daterange(p_fecha_inicio, p_fecha_fin, '[')  
    )  
    ORDER BY h.piso, h.numero;  
END;  
$$;  
-- COMMENT ON FUNCTION fn_habitaciones_disponibles(DATE,DATE)  
-- IS 'Lista las habitaciones libres en el rango dado.';
```

7.2 Tasa Ocupación

Tasa de ocupacion (%) del hotel en un rango dado (sobre todas las habitaciones del establecimiento).

```
CREATE OR REPLACE FUNCTION fn_tasa_ocupacion (  
    p_fecha_inicio DATE,  
    p_fecha_fin    DATE  
)  
RETURNS NUMERIC  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_dias          INT;  
    v_habitaciones  INT;  
    v_noches_ocupadas INT;  
BEGIN  
    v_dias := p_fecha_fin - p_fecha_inicio;  
    IF v_dias <= 0 THEN  
        RAISE EXCEPTION 'Rango invalido';  
    END IF;  
  
    SELECT COUNT(*) INTO v_habitaciones FROM habitacion;  
  
    SELECT COALESCE(SUM(  
        LEAST(r.fecha_fin, p_fecha_fin)  
        - GREATEST(r.fecha_inicio, p_fecha_inicio)  
    ), 0)  
    INTO v_noches_ocupadas  
    FROM reservacion r  
    WHERE r.estado IN ('CONFIRMADA', 'CHECK_IN', 'CHECK_OUT')  
        AND r.fecha_inicio < p_fecha_fin  
        AND r.fecha_fin    > p_fecha_inicio;  
  
    IF v_habitaciones = 0 THEN RETURN 0; END IF;  
  
    RETURN ROUND(100.0 * v_noches_ocupadas / (v_habitaciones * v_dias), 2);  
END;  
$;
```

7.3 Función auxiliar: total de una estancia

Devuelve el costo de hospedaje (noches × tarifa del tipo) más la suma de los servicios consumidos. Es reutilizada por el procedimiento de check-out.

```
CREATE OR REPLACE FUNCTION fn_total_estancia (
    p_id_estancia INT,
    p_iva          NUMERIC DEFAULT 0.21
)
RETURNS TABLE (
    noches          INT,
    subtotal_habitacion NUMERIC,
    subtotal_servicios NUMERIC,
    impuestos       NUMERIC,
    total           NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_noches    INT;
    v_precio    NUMERIC;
    v_sub_hab   NUMERIC;
    v_sub_serv  NUMERIC;
    v_imp       NUMERIC;
BEGIN
    SELECT (r.fecha_fin - r.fecha_inicio), th.precio
        INTO v_noches, v_precio
        FROM check_in_check_out e
        JOIN reservacion r      ON r.id_reserva = e.id_reserva
        JOIN habitacion h       ON h.id_habitacion = e.id_habitacion
        JOIN tipo_habitacion th ON th.id_tipo = h.id_tipo
        WHERE e.id_estancia = p_id_estancia;

    IF v_noches IS NULL THEN
        RAISE EXCEPTION 'No existe la estancia %', p_id_estancia;
    END IF;

    v_sub_hab := v_noches * v_precio;

    SELECT COALESCE(SUM(cs.subtotal), 0)
        INTO v_sub_serv
        FROM consumo_servicio cs
        WHERE cs.id_estancia = p_id_estancia;

    v_imp := ROUND((v_sub_hab + v_sub_serv) * p_iva, 2);

    noches          := v_noches;
    subtotal_habitacion := v_sub_hab;
    subtotal_servicios := v_sub_serv;
    impuestos       := v_imp;
    total           := v_sub_hab + v_sub_serv + v_imp;
    RETURN NEXT;
END;
$$;
-- COMMENT ON FUNCTION fn_total_estancia(INT, NUMERIC)
-- IS 'Calcula importes de la estancia: habitacion + servicios + IVA.';
```

7.4 Procedimiento almacenado: check-out

Realiza el proceso de check-out: valida la estancia, evita la doble facturación, calcula el total (hospedaje + servicios + impuestos), registra la salida real, genera la factura y marca la reservación como finalizada. Todo se ejecuta en una sola transacción.

```
CREATE OR REPLACE FUNCTION fn_check_solape_reserva()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_conflicto INT;
BEGIN
    IF NEW.estado NOT IN ('PENDIENTE', 'CONFIRMADA', 'CHECK_IN') THEN
        RETURN NEW;
    END IF;

    IF NEW.fecha_fin <= NEW.fecha_inicio THEN
        RAISE EXCEPTION 'fecha_fin (%) debe ser posterior a fecha_inicio (%)',
            NEW.fecha_fin, NEW.fecha_inicio;
    END IF;

    SELECT r.id_reserva INTO v_conflicto
    FROM reservacion r
    WHERE r.id_habitacion = NEW.id_habitacion
        AND r.estado IN ('PENDIENTE', 'CONFIRMADA', 'CHECK_IN')
        AND r.id_reserva <> COALESCE(NEW.id_reserva, -1)
        AND daterange(r.fecha_inicio, r.fecha_fin, '[')
        && daterange(NEW.fecha_inicio, NEW.fecha_fin, '[')
    LIMIT 1;

    IF v_conflicto IS NOT NULL THEN
        RAISE EXCEPTION
            'Conflicto: la habitacion % ya tiene la reserva % activa entre % y %.'
            NEW.id_habitacion, v_conflicto, NEW.fecha_inicio, NEW.fecha_fin;
    END IF;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS tg_reserva_sin_solape ON reservacion;
CREATE TRIGGER tg_reserva_sin_solape
BEFORE INSERT OR UPDATE OF id_habitacion, fecha_inicio, fecha_fin, estado
ON reservacion
FOR EACH ROW EXECUTE FUNCTION fn_check_solape_reserva();
```

7.5 Trigger de validación de fechas de estancia

sp_registrar_reserva cuenta con un bloqueo por transacciones (FOR UPDATE) para evitar las dobles reservas simultáneas (condición de carrera).

```
-- =====
-- TRIGGER extra 2: Validar que fecha_salida no sea menor a fecha_entrada
-- =====
CREATE OR REPLACE FUNCTION fn_trg_validar_fechas_estancia()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha_salida_real IS NOT NULL AND NEW.fecha_salida_real::date < NEW.fecha_entrada_real::date THEN
        RAISE EXCEPTION 'Error: La fecha de salida (%) no puede ser menor a la fecha de entrada (%)',
            NEW.fecha_salida_real, NEW.fecha_entrada_real;
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_validar_estancia ON check_in_check_out;
CREATE TRIGGER trg_validar_estancia
BEFORE INSERT OR UPDATE ON check_in_check_out
FOR EACH ROW
EXECUTE FUNCTION fn_trg_validar_fechas_estancia();
```

8. Instrucciones de Ejecución

Requisito: PostgreSQL 17 o superior. Los scripts están comentados e indican el propósito de cada sección.

8.1 Archivos entregables

Archivo	Contenido
ProyectoBD.erdplus	Diagrama ER (notacion Chen) corregido, editable en ERDPlus
Diagrama_ER_Chen.png	Imagen del diagrama ER
Tablas.sql	Creacion de base de datos, tablas y restricciones
Poblado.sql	Carga inicial de datos
Consultas.sql	Consultas SQL basicas y avanzadas
Programacion.sql	Funciones, procedimientos y triggers
Documento_BDD.docx	Este documento